

ETHEREUM

DEEP-DIVE RESEARCH SERIES

FILE 1 OF 3

ORIGINS & TECHNOLOGY

Founding Story · Team Biographies · The World Computer Vision
EVM · Smart Contracts · The Merge · Gas System · Transaction Lifecycle

Prepared by: NASIR BELLO

June 2025 · Personal Research Report · For Informational Purposes Only

TABLE OF CONTENTS

01	Executive Summary	3
02	Founding Story & History	4
	2.1 Vitalik Buterin & the Bitcoin Magazine Years	4
	2.2 The Ethereum Whitepaper — November 2013	4
	2.3 Gavin Wood & the Yellow Paper	5
	2.4 The Ethereum Foundation & Swiss Non-Profit	5
	2.5 The 2014 Crowdsale	6
	2.6 From Idea to Mainnet — Frontier (2015)	6
03	Founding Team Biographies	7
04	The Problem Ethereum Set Out to Solve	11
	4.1 Bitcoin's Limitations	11
	4.2 The World Computer Vision	11
	4.3 Smart Contracts — What They Are	12
	4.4 The EVM Concept	12
05	How Ethereum Actually Works	13
	5.1 Accounts & State Model	13
	5.2 The Ethereum Virtual Machine (EVM)	14
	5.3 Smart Contracts & Solidity	15
	5.4 Original Proof of Work (PoW)	16
	5.5 The Merge — Transition to Proof of Stake	17
	5.6 Gasper Consensus (LMD-GHOST + Casper FFG)	18
	5.7 The Gas System Explained	19
	5.8 Full Transaction Lifecycle	20
	5.9 Nodes & Client Diversity	21
06	Validators, Staking & Block Production	22

01 — EXECUTIVE SUMMARY

Ethereum is the world's most widely used programmable blockchain — a decentralised platform that enables developers to build and deploy applications that run exactly as programmed, without any possibility of downtime, censorship, fraud, or third-party interference. Since its launch in July 2015, Ethereum has grown from an experimental project conceived by a teenage programmer into the foundational infrastructure of the global decentralised finance (DeFi) ecosystem, NFT markets, and Web3 applications.

Where Bitcoin introduced the concept of decentralised, trustless money, Ethereum extended that idea to **decentralised, trustless computation**. The Ethereum Virtual Machine (EVM) — a global, shared computer running across thousands of nodes — allows anyone to deploy programs called **smart contracts** that execute automatically when predefined conditions are met, without any central authority. This single innovation unlocked an entirely new category of financial and social applications that had never previously been possible.

This report — the first of three in the Ethereum Deep-Dive Research Series — covers everything from the founding story and the eight co-founders who built Ethereum, to a comprehensive technical breakdown of how the network actually works, including the historic transition from Proof of Work to Proof of Stake known as **The Merge**.

METRIC	VALUE
Founded	2013 (whitepaper) · 2014 (foundation) · 2015 (mainnet)
Mainnet Launch	July 30, 2015 (Frontier)
Consensus Mechanism	Proof of Stake (post-Merge, Sept 2022)
Native Token	ETH (Ether)
Base Layer TPS	~15-30 TPS (base) · 100,000+ with L2s
Average Block Time	~12 seconds
Active Validators	800,000+ (largest PoS validator set)
Total ETH Staked	~32 million ETH (~27% of supply)
Programming Language	Solidity, Vyper (smart contracts)
EVM Compatible Chains	50+ (Polygon, BSC, Avalanche C-Chain, etc.)
All-Time High Price	~\$4,878 (Nov 2021)

02 — FOUNDING STORY & HISTORY

2.1 — Vitalik Buterin & the Bitcoin Magazine Years

The story of Ethereum begins with a Russian-Canadian prodigy named **Vitalik Buterin**, born in Kolomna, Russia in 1994 and raised in Canada from age six. A child gifted in mathematics, economics, and programming, Buterin discovered Bitcoin in 2011 at age 17 through his father, Dmitry Buterin — himself a computer scientist.

Buterin was immediately captivated by Bitcoin's potential — not just as a currency, but as a programmable system. In 2011, he co-founded **Bitcoin Magazine**, one of the first serious publications dedicated to cryptocurrency journalism and analysis. Writing prolifically for the magazine, Buterin developed an unusually deep understanding of Bitcoin's architecture — and its limitations. He spent two years attempting to add programmability to Bitcoin by working on various projects that tried to extend Bitcoin's scripting capabilities. Every approach hit the same wall: Bitcoin's scripting language was deliberately limited, and the Bitcoin community was deeply conservative about protocol changes.

2.2 — The Ethereum Whitepaper — November 2013

In November 2013, at just **19 years old**, Vitalik Buterin wrote and circulated the Ethereum whitepaper — titled *'Ethereum: A Next-Generation Smart Contract and Decentralised Application Platform'*. Rather than trying to add features to Bitcoin, Buterin proposed building an entirely new blockchain from scratch — one designed from the ground up for general-purpose programmability.

The whitepaper described a Turing-complete blockchain — a system capable of executing any computation that a traditional computer could perform. The key insight was deceptively simple: instead of building specific features (like decentralised exchange or decentralised DNS) directly into the blockchain protocol, build a general-purpose programmable layer and let developers build any application on top of it. This concept — the **smart contract platform** — would become the defining blockchain innovation of the decade.

The Whitepaper's Central Thesis

Bitcoin has a scripting language, but it is not Turing-complete — it deliberately cannot perform loops or complex computations to avoid infinite loops that could crash nodes.

Buterin's proposal: build a blockchain with a built-in Turing-complete programming language. Developers could write 'smart contracts' — self-executing programs stored on-chain — that could encode any rule, condition, or financial logic imaginable.

The whitepaper circulated among the Bitcoin developer community in late 2013. Within weeks, dozens of talented developers had reached out to Buterin, eager to help build it.

2.3 — Gavin Wood & the Yellow Paper

Among those who read Buterin's whitepaper and immediately reached out was **Gavin Wood**, a British computer scientist with a PhD from the University of York and extensive experience in programming language design. Wood joined the project in early 2014 and made two contributions that were arguably as important as the whitepaper itself.

- **The Yellow Paper:** Wood authored the Ethereum Yellow Paper — the formal technical specification of the Ethereum Virtual Machine. While Buterin's whitepaper described the concept, Wood's Yellow Paper described the precise mathematical and computational rules that defined exactly how Ethereum would work at the byte level. This document became the definitive technical reference for anyone implementing an Ethereum client.
- **Solidity:** Wood invented **Solidity** — the high-level programming language for writing Ethereum smart contracts. Solidity's JavaScript-like syntax made smart contract development accessible to the millions of web developers already familiar with similar languages. It remains the dominant smart contract language in blockchain to this day.

2.4 — The Ethereum Foundation & Swiss Non-Profit

In June 2014, the **Ethereum Foundation** was established as a non-profit organisation in Zug, Switzerland — a location chosen for its favourable regulatory environment for blockchain entities (Zug is known as 'Crypto Valley'). The Foundation's mandate was to support the research, development, and promotion of the Ethereum platform and its ecosystem.

The choice of a non-profit structure was deliberate and philosophically important. Unlike a for-profit company (which might be pressured to compromise the protocol for shareholder returns), a foundation structure signalled that Ethereum was intended to be a public good — infrastructure for the world, not a product owned by investors. The Ethereum Foundation continues to fund core research and development to this day, though Ethereum's development has become increasingly decentralised across dozens of independent client teams.

2.5 — The 2014 Crowdsale

To fund Ethereum's development, the Foundation conducted a 42-day public crowdsale from **July 22 to September 2, 2014** — one of the first major token sales in cryptocurrency history. Participants could purchase ETH (then called 'Ether') using Bitcoin at a rate of 2,000 ETH per Bitcoin for the first 14 days, declining to 1,337 ETH per Bitcoin by the end.

CROWDSALE METRIC	VALUE
Duration	42 days (July 22 — Sept 2, 2014)
ETH Sold	~60 million ETH
Bitcoin Raised	~31,529 BTC

USD Value at Time	~\$18.4 million
Initial ETH Price	~\$0.31 per ETH
Retained by Foundation	~9.9% of initial supply for development
Retained by Early Contributors	~9.9% for team and early devs

The crowdsale raised approximately \$18.4 million — a massive amount for a blockchain project in 2014. It established the initial distribution of ETH and provided the Foundation with multi-year runway to build the protocol. The crowdsale participants who bought ETH at \$0.31 and held through to the 2021 all-time high of \$4,878 achieved a return of over 15,000x.

2.6 — From Idea to Mainnet: Frontier (2015)

Ethereum's development followed a carefully planned roadmap from the outset, divided into four named phases — **Frontier**, **Homestead**, **Metropolis**, and **Serenity**. Each phase represented a major protocol upgrade with progressively more features and stability.

PHASE	DATE	KEY FEATURES
Frontier	July 2015	Bare-bones mainnet launch — command line only, for developers
Homestead	March 2016	First stable release — usable by non-technical users
Metropolis	2017-2019	Byzantium + Constantinople upgrades — privacy, efficiency
Serenity	2020-2022+	Beacon Chain, The Merge, Proof of Stake transition

Ethereum's Frontier mainnet launched on **July 30, 2015** — nearly two years after the whitepaper. The initial launch was deliberately minimal: a command-line interface with no user-friendly tools, intended for developers to begin experimenting with smart contracts. Mining began immediately, and the first Ether was distributed to miners and crowdsale participants. Despite its rough edges, the community response was electric — within weeks, developers around the world were deploying the first smart contracts in history.

03 — FOUNDING TEAM BIOGRAPHIES

Ethereum is unique in blockchain history for having **eight co-founders** — an unusually large founding team whose diverse backgrounds in computer science, economics, law, and entrepreneurship shaped Ethereum's multidimensional identity. Many have gone on to found their own major blockchain projects.

Vitalik Buterin

Co-Founder & Chief Scientist

Vitalik Buterin is widely regarded as one of the most important figures in the history of blockchain technology. Born in Russia in 1994 and raised in Canada, he demonstrated extraordinary mathematical and programming ability from a young age. After co-founding Bitcoin Magazine at 17 and spending two years immersed in the Bitcoin community, he wrote the Ethereum whitepaper at 19 — an achievement that remains one of the most remarkable intellectual contributions in technology history. Buterin never completed his undergraduate degree at the University of Waterloo, instead dropping out in 2014 to work on Ethereum full-time after receiving a \$100,000 Thiel Fellowship. He remains the most influential voice in Ethereum's development, authoring Ethereum Improvement Proposals (EIPs), writing extensively on cryptography, game theory, and blockchain design on his blog, and representing Ethereum at major global forums. Despite offers that would have made him a billionaire many times over, Buterin is known for his frugality and genuine commitment to Ethereum as a public good rather than a personal wealth vehicle.

Gavin Wood

Co-Founder, CTO & Yellow Paper Author

Dr. Gavin Wood holds a PhD in Computer Science from the University of York, where his research focused on music visualisation — far from blockchain, but deeply connected to the intersection of mathematics and creative systems. After joining Ethereum in early 2014, Wood made two foundational contributions: the Ethereum Yellow Paper (the formal technical specification) and the Solidity programming language. Wood served as Ethereum's CTO until 2016, when he departed to pursue his own vision of a blockchain ecosystem. He founded **Parity Technologies** (now Parity Technologies), which built one of the most important Ethereum clients (Parity/OpenEthereum), and then founded **Polkadot** — a multi-chain blockchain network designed to enable interoperability between blockchains. He also created the Web3 Foundation to fund Polkadot's development. Wood is credited with coining the term **'Web3'** — the vision of a decentralised internet — in 2014, a concept that would define the next decade of blockchain discourse.

Joseph Lubin

Co-Founder & ConsenSys Founder

Joseph Lubin is the most entrepreneurially prolific of Ethereum's co-founders. With a background in computer science and cognitive science from Princeton, and a career spanning software engineering, robotics, and traditional finance (including a role at Goldman Sachs), Lubin brought business and organisational acumen to a team of brilliant but inexperienced technologists. After helping establish the Ethereum Foundation, Lubin founded **ConsenSys** in 2014 — a Brooklyn-based blockchain software company that became the largest organisation in the Ethereum ecosystem. ConsenSys built and/or invested in many of Ethereum's most critical tools: MetaMask (the most used Ethereum wallet, with 30M+ users), Infura (the API provider that millions of dApps use to connect to Ethereum), Truffle (the leading smart contract development framework), and dozens of other ecosystem projects. ConsenSys is arguably responsible for more of Ethereum's practical infrastructure than any other single organisation outside the Ethereum Foundation itself.

Charles Hoskinson

Co-Founder & Cardano Founder

Charles Hoskinson is perhaps the most controversial of Ethereum's co-founders — and the one whose departure from the project led to the most significant downstream impact on the blockchain industry. An American mathematician and entrepreneur, Hoskinson co-founded Ethereum but departed in 2014 after a fundamental disagreement with Vitalik Buterin over whether Ethereum should remain a non-profit or become a for-profit company. Hoskinson wanted the for-profit route; Buterin and others insisted on the non-profit foundation model. After leaving Ethereum, Hoskinson co-founded IOHK (Input Output Hong Kong) and began building **Cardano** — a blockchain that positions itself as a more academically rigorous and formally verified alternative to Ethereum. Cardano's development is driven by peer-reviewed academic research, and it has become one of the top 10 cryptocurrencies by market cap. Hoskinson and Buterin have maintained a public rivalry, with Hoskinson frequently critical of Ethereum's technical decisions and governance.

Anthony Di Iorio

Co-Founder

Anthony Di Iorio is a Canadian entrepreneur and one of the earliest adopters of Bitcoin in Canada. He provided critical early funding for Ethereum's development and co-founded the project before formally organising the Ethereum Foundation. Di Iorio hosted the first Ethereum development session in his Toronto apartment in late 2013, bringing together the early founding team. Di Iorio later founded **Jaxx** (now Jaxx Liberty) — a popular multi-platform cryptocurrency wallet — and Decentral, a blockchain innovation hub in Toronto. In 2021, Di Iorio made headlines by announcing he was selling his crypto holdings and stepping back from the industry, citing safety concerns related to his high public profile in the crypto space. His early financial support and organizational contributions were essential to getting Ethereum off the ground.

Mihai Alisie

Co-Founder & Bitcoin Magazine Co-Founder

Mihai Alisie is a Romanian entrepreneur who co-founded Bitcoin Magazine alongside Vitalik Buterin in 2011, making him one of the earliest people to recognise Buterin's potential. When Buterin began working on Ethereum, Alisie was one of the first he brought into the project. Alisie helped establish the Ethereum Foundation in Zug, Switzerland — handling much of the legal and organisational groundwork that made the Foundation's structure possible. Alisie later founded **AKASHA** — a decentralised social media platform built on Ethereum — exploring one of the original use cases Ethereum was designed to enable. He has remained a consistent voice for Ethereum's potential to enable censorship-resistant social communication.

Jeffrey Wilcke

Co-Founder & Go-Ethereum (Geth) Lead

Jeffrey Wilcke is a Dutch developer who wrote the first Go language implementation of Ethereum — known as **Go-Ethereum (Geth)**. Today, Geth is the most widely used Ethereum client, running on the majority of Ethereum nodes worldwide. Wilcke's implementation was not just a translation of the Yellow Paper into code — it established architectural patterns and design decisions that define how Ethereum works at the software level to this day. Wilcke led the go-ethereum team at the Ethereum Foundation until 2018, when he departed to found Grid Games — a gaming studio exploring blockchain-based game mechanics. Despite his quieter public profile compared to other co-founders, Wilcke's technical contribution through Geth is embedded in the infrastructure of the entire Ethereum ecosystem.

Amir Chetrit

Co-Founder

Amir Chetrit is the least publicly prominent of Ethereum's eight co-founders. An Israeli entrepreneur with early involvement in the Bitcoin community, Chetrit was involved in Ethereum's early conceptual and organisational phase. He co-founded the project alongside the other seven founders but stepped back from active involvement relatively early in the development process. He has maintained a low public profile in subsequent years and has not founded any major follow-on blockchain projects.

04 — THE PROBLEM ETHEREUM SET OUT TO SOLVE

4.1 — Bitcoin's Limitations as Programmable Money

To understand Ethereum's purpose, you must first understand what Bitcoin cannot do — and why. Bitcoin is a masterpiece of minimalist design. Its scripting language is deliberately limited: it can verify signatures, check time locks, and handle basic multi-signature conditions — but it cannot perform loops, store complex state, or execute arbitrary logic. This limitation was intentional. Satoshi Nakamoto designed Bitcoin to do one thing exceptionally well: transfer value securely without a trusted third party.

- **No Turing-completeness:** Bitcoin scripts cannot loop or perform complex computations. Any application logic must be built off-chain.
- **No native state:** Bitcoin has no concept of 'accounts' with balances that persist between transactions. Everything is based on unspent transaction outputs (UTXOs).
- **No application layer:** Building a decentralised exchange, lending protocol, or prediction market on Bitcoin requires enormous off-chain infrastructure that reintroduces centralisation.
- **Conservative governance:** The Bitcoin community is deliberately slow to add features. Proposals to extend Bitcoin's programmability (like Coloured Coins or Counterparty) were consistently rejected or unsupported.

4.2 — The World Computer Vision

Buterin's vision for Ethereum was captured in a phrase that became the project's defining metaphor: the **'World Computer'**. Where Bitcoin was digital gold — a store of value — Ethereum would be a decentralised computing platform. Just as the internet democratised information publishing, Ethereum would democratise application deployment: anyone, anywhere, could deploy a program that would run permanently, without censorship, on a global shared computer.

The World Computer Concept

Imagine a computer that no one owns and everyone can use.

A computer that runs exactly as programmed — no one can shut it down, alter its outputs, or prevent anyone from using it.

A computer whose code and data are publicly auditable by anyone on Earth.

This is what Ethereum set out to build — and what smart contracts make possible.

4.3 — Smart Contracts: What They Are and Why They Matter

The term '**smart contract**' was coined by cryptographer Nick Szabo in 1994 — nearly two decades before Ethereum. Szabo defined a smart contract as a computerised transaction protocol that executes the terms of a contract automatically. Ethereum was the first platform to make this practically implementable at scale.

A smart contract on Ethereum is simply a program stored on the blockchain. Once deployed, it runs exactly as coded — no one (not even its creator) can modify or stop it unless it is programmed to allow modification. It has its own Ethereum address, can hold ETH and other tokens, and executes its functions when triggered by transactions.

TRADITIONAL CONTRACT	SMART CONTRACT
Enforced by courts and legal systems	Enforced by code and consensus
Requires intermediaries (lawyers, banks)	Self-executing — no intermediaries needed
Takes days/weeks to settle	Settles in seconds
Can be interpreted ambiguously	Executes exactly as written
Expensive — legal fees	Cheap — only gas fees
Jurisdiction-dependent	Borderless and permissionless
Can be breached or ignored	Cannot be stopped once deployed

4.4 — The EVM Concept

To run smart contracts on a decentralised network of thousands of computers — all of which must agree on the result — Ethereum needed a standardised computing environment. The solution was the **Ethereum Virtual Machine (EVM)**: a sandboxed, deterministic virtual computer that runs on every Ethereum node simultaneously.

- **Deterministic:** Given the same input and the same starting state, the EVM always produces exactly the same output on every node in the world — no matter what hardware or operating system the node runs on.
- **Sandboxed:** Smart contracts run in a completely isolated environment. They cannot access the node's file system, network, or any external data (unless provided via oracles).
- **Turing-complete:** The EVM can execute any computation that a traditional computer can — including loops, conditionals, and complex data structures.
- **Gas-metered:** Every operation the EVM performs costs a defined amount of 'gas' — preventing infinite loops and ensuring computational resources are fairly priced.

05 — HOW ETHEREUM ACTUALLY WORKS

Ethereum's architecture is substantially more complex than Bitcoin's — by design. It must support not just value transfer but arbitrary computation, persistent application state, and a programmable environment that thousands of developers can build on simultaneously. Here is a complete breakdown of how every layer of the system works.

5.1 — Accounts & the State Model

Unlike Bitcoin's UTXO model (where value is tracked as unspent outputs from previous transactions), Ethereum uses an **account-based model** — much closer to how traditional bank accounts work. There are two types of accounts:

Externally Owned Accounts (EOAs)

- Controlled by a private key held by a human (or software).
- Has an ETH balance and a transaction nonce (a counter preventing replay attacks).
- Can initiate transactions — sending ETH or calling smart contracts.
- Has no associated code — it is just an address and a balance.

Contract Accounts

- Created when a smart contract is deployed to the network.
- Has an ETH balance, a nonce, and associated **bytecode** (compiled smart contract code).
- Has **storage** — a persistent key-value database that the contract can read from and write to.
- Can only execute code when triggered by a transaction from an EOA (or another contract).
- Cannot initiate transactions on its own — it can only respond to calls.

The Global State

Ethereum's **global state** is the complete snapshot of all account balances and all contract storage at a given block. Every transaction modifies this global state, and every Ethereum node maintains an identical copy of it. The state is stored as a **Merkle Patricia Trie** — a cryptographic data structure that makes it possible to efficiently prove the state of any account without downloading the entire database.

5.2 — The Ethereum Virtual Machine (EVM)

The EVM is the engine that executes smart contract code. It is a **stack-based virtual machine** — meaning it operates on a last-in, first-out stack of 256-bit values (a design choice that makes it easier to handle Ethereum's 256-bit addresses and cryptographic operations).

How the EVM Executes Code

- **Bytecode:** Smart contracts are written in high-level languages (Solidity, Vyper) and compiled into EVM bytecode — a sequence of low-level instructions (opcodes) that the EVM can execute.
- **Opcodes:** The EVM has ~150 defined opcodes — basic operations like ADD, MULTIPLY, STORE (write to storage), LOAD (read from storage), CALL (call another contract), and RETURN.
- **Memory:** The EVM has a temporary memory space (cleared after each transaction) for intermediate calculations, separate from the persistent contract storage.
- **Stack:** All arithmetic and logical operations work by pushing values onto and popping values off a 1,024-element stack.
- **Determinism:** The EVM has no access to random numbers, the current time (except the block timestamp, which validators can slightly manipulate), or external data — ensuring every node produces identical results.

EVM Compatibility — The Network Effect

One of Ethereum's most powerful competitive advantages is EVM compatibility. Because the EVM is an open standard, dozens of other blockchains have built EVM-compatible execution environments — meaning Ethereum smart contracts can run on Polygon, BNB Chain, Avalanche's C-Chain, Arbitrum, Optimism, and many others with minimal or no code changes. This has created a massive network effect: the tools, developer knowledge, and audit firms built for Ethereum's EVM serve the entire EVM-compatible ecosystem.

5.3 — Smart Contracts & Solidity

Solidity is Ethereum's primary smart contract programming language, designed by Gavin Wood and developed by the Ethereum team. It is a statically typed, object-oriented language with syntax inspired by JavaScript and C++, making it relatively accessible to web developers.

Key Solidity Concepts

- **Contract:** The fundamental unit of Solidity code — similar to a class in object-oriented programming. A contract has state variables (persistent storage), functions (callable logic), and events (logs emitted to the blockchain).
- **State Variables:** Variables stored permanently in contract storage on the blockchain. Every read and write to state variables costs gas.
- **Functions:** Executable code within a contract. Functions can be public (callable by anyone), private (only callable within the contract), or internal (callable by the contract and its children).
- **Modifiers:** Reusable conditions that can be attached to functions — e.g., 'onlyOwner' which ensures only the contract owner can call a function.
- **Events:** Logs emitted by contracts that are stored on the blockchain and can be indexed and queried by off-chain applications.

- **Mappings:** Key-value data structures used for storing balances, ownership records, and other associative data.

Other Smart Contract Languages

- **Vyper:** A Python-like language for Ethereum smart contracts, designed to be simpler and more auditable than Solidity. Preferred for security-critical applications.
- **Yul / Yul+:** An intermediate language for writing highly optimised, low-level EVM code.
- **Huff:** An ultra-low-level language for writing maximally gas-efficient contracts.

5.4 — Original Proof of Work (PoW)

From its launch in 2015 until September 2022, Ethereum used **Proof of Work (PoW)** consensus — the same mechanism pioneered by Bitcoin. Under PoW, miners competed to solve a computationally difficult mathematical puzzle (finding a hash below a target value) by trying billions of random inputs per second. The first miner to find a valid solution broadcast the new block to the network and received a block reward in ETH.

Ethereum's PoW algorithm was called **Ethash** — deliberately designed to be memory-hard (requiring large amounts of RAM) rather than pure compute-intensive, which was intended to resist ASIC mining and keep mining accessible to GPU miners. Despite this design, large GPU mining farms dominated Ethereum mining in practice.

- **Energy consumption:** At its peak, Ethereum PoW consumed approximately 78 TWh of electricity per year — comparable to a medium-sized country.
- **Mining centralisation:** GPU mining farms concentrated in regions with cheap electricity gave those operators a systematic advantage.
- **Security cost:** The security of PoW comes from the cost of hardware and electricity — an expensive and wasteful security mechanism.
- **Limited scalability:** PoW's slow block times (targeted ~13 seconds) and difficulty adjustment mechanism created an inherent TPS ceiling.

5.5 — The Merge: Transition to Proof of Stake

The transition from Proof of Work to Proof of Stake — known as '**The Merge**' — was the most complex and consequential upgrade in Ethereum's history, and arguably the most significant technical event in all of blockchain history. It was completed on **September 15, 2022**, and it reduced Ethereum's energy consumption by approximately **99.95%** overnight.

How The Merge Worked

The Merge was not a simple switch. It required years of preparation and a novel technical approach: running two parallel chains simultaneously and then merging them.

- **The Beacon Chain (December 2020):** Ethereum launched a separate PoS chain — the Beacon Chain — in December 2020. The Beacon Chain ran alongside the original PoW chain but processed no transactions. Its sole purpose was to test and validate the PoS consensus mechanism with real stakes and real validators.
- **18 months of parallel operation:** From December 2020 to September 2022, both chains ran simultaneously. Over 400,000 validators staked ETH on the Beacon Chain, demonstrating the PoS system worked under real economic conditions.
- **The Merge event:** On September 15, 2022, the original PoW execution layer 'merged' with the Beacon Chain consensus layer. The PoW chain stopped producing blocks at a specific Total Difficulty threshold, and the Beacon Chain's validators took over block production instantaneously.
- **Zero downtime:** The transition happened without any interruption to Ethereum's operation. Transactions continued processing through the merge event without pause.

The Merge's Impact

Energy reduction: ~99.95% less electricity consumption — from ~78 TWh/year to ~0.01 TWh/year.

Issuance reduction: ETH issuance dropped by ~90% after The Merge, as validator rewards are far lower than miner rewards were.

Security model change: Ethereum's security now comes from economic stake rather than physical hardware — attackers must acquire 33%+ of staked ETH to disrupt consensus.

The Merge was completed without a single major incident — a remarkable engineering achievement given the complexity of the operation.

5.6 — Gasper Consensus: LMD-GHOST + Casper FFG

Post-Merge Ethereum uses a consensus mechanism called **Gasper** — a combination of two algorithms: **LMD-GHOST** (for fork choice) and **Casper FFG** (for finality). Understanding both is essential to understanding how Ethereum achieves agreement.

LMD-GHOST — Fork Choice Rule

LMD-GHOST stands for 'Latest Message Driven Greedy Heaviest Observed SubTree.' When validators see multiple competing chain tips (forks), LMD-GHOST determines which fork to follow by selecting the fork that has the most validator attestation weight behind it — specifically, the most recent vote (latest message) from each validator, counting only votes for blocks in the current subtree. This ensures the chain converges on the fork supported by the most staked ETH.

Casper FFG — Finality Gadget

LMD-GHOST alone provides probabilistic finality — the chain can always be reorganised if enough validators switch. Casper FFG adds **economic finality**: once 2/3 of validators have attested to a checkpoint, that checkpoint becomes **finalised** — meaning it can never be reverted without destroying at least 1/3 of all staked

ETH (tens of billions of dollars). Finality in Ethereum occurs approximately every **2 epochs (~12.8 minutes)**.

Slots and Epochs

TERM	DURATION	DESCRIPTION
Slot	12 seconds	One block opportunity. A validator is randomly selected to propose a block each slot.
Epoch	32 slots (~6.4 min)	A set of 32 slots. At the end of each epoch, validators vote on checkpoints for Casper FFG.
Finality	~2 epochs (~12.8 min)	Once 2/3 of validators attest to an epoch boundary, blocks in that epoch are finalised.

5.7 — The Gas System Explained

Gas is Ethereum's unit of computational work. Every operation the EVM performs — adding two numbers, reading from storage, calling another contract — has a defined gas cost. Users pay for gas using ETH, and the gas system serves two critical functions: it prevents infinite loops (every loop iteration costs gas, so loops always terminate when gas runs out) and it creates a market for blockspace (users bid for inclusion via gas fees).

EIP-1559: The Fee Market Reform (August 2021)

Before August 2021, Ethereum used a simple auction system where users set a 'gas price' and miners selected transactions with the highest prices. This was unpredictable and created wild fee spikes. **EIP-1559** replaced this with a two-component fee structure:

- **Base Fee:** A protocol-determined fee that every transaction must pay. The base fee adjusts automatically based on block fullness — if blocks are more than 50% full, the base fee increases by up to 12.5% for the next block; if less than 50% full, it decreases. Crucially, the **base fee is burned** — destroyed permanently — making ETH deflationary when network usage is high.
- **Priority Fee (Tip):** An optional additional payment from the user directly to the validator (formerly the miner) as an incentive for faster inclusion. Users set a 'max priority fee' they are willing to pay.
- **Max Fee:** The absolute maximum the user is willing to pay per gas unit (base fee + priority fee). Any difference between the max fee and (base fee + priority fee) is refunded to the user.

Gas Costs for Common Operations

OPERATION	GAS COST	USD EQUIVALENT (ETH @ \$3,000)
Simple ETH transfer	21,000 gas	~\$0.50-\$5 (varies by base fee)
ERC-20 token transfer	~65,000 gas	~\$1.50-\$15
Uniswap V3 swap	~150,000 gas	~\$4-\$40
NFT mint (simple)	~100,000 gas	~\$3-\$30

Deploy a contract	~500,000+ gas	~\$15-\$150+
Complex DeFi interaction	~300,000+ gas	~\$9-\$90+

5.8 — Full Transaction Lifecycle

Here is how a transaction moves through the Ethereum network from submission to finality:

1	User Creates & Signs Transaction A user specifies: recipient address, ETH value, data (for contract calls), gas limit (max gas units willing to use), max fee per gas, and max priority fee. The transaction is signed with the user's private key using ECDSA cryptography.
2	Broadcast to the Mempool The signed transaction is broadcast to the Ethereum peer-to-peer network. Each node that receives it validates the signature and checks the sender has sufficient ETH, then adds it to its local mempool — a waiting room of pending transactions.
3	Validator Selection At the start of each 12-second slot, a validator is pseudo-randomly selected to propose the next block. Selection probability is proportional to the validator's effective balance (up to 32 ETH).
4	Block Building The selected validator (or a MEV-Boost block builder) selects transactions from the mempool — prioritising those with the highest priority fees — and assembles them into a block. The block cannot exceed the gas limit (~30 million gas per block).
5	EVM Execution Each transaction in the block is executed by the EVM in sequence. For simple ETH transfers, this is trivial. For smart contract calls, the EVM runs the contract bytecode, modifying account balances and contract storage as dictated by the code.
6	Block Proposal & Attestation The proposing validator broadcasts the new block to the network. Other validators in the current slot's committee verify the block (check signatures, re-execute transactions, verify state root) and broadcast their attestations — votes confirming the block is valid.
7	Inclusion in Chain Once the block has sufficient attestations, it is included in the canonical chain. The transaction has now achieved 'inclusion' — it is in the chain but not yet finalised.

8 Probabilistic Finality (~1-2 epochs)

As subsequent blocks build on top of the block containing the transaction, reverting it becomes progressively more expensive. After 2 epochs (~12.8 minutes), the block containing the transaction is finalised by Casper FFG — it can never be reverted without destroying billions of dollars of staked ETH.

5.9 — Nodes & Client Diversity

Ethereum's post-Merge architecture separates the blockchain into two distinct layers, each requiring its own client software:

Execution Layer Clients

- **Go-Ethereum (Geth):** The original client written by Jeffrey Wilcke. Still the most widely used execution client (~55% of nodes). Written in Go.
- **Nethermind:** A high-performance C# implementation popular among enterprise operators.
- **Besu:** A Java implementation by Hyperledger, popular for enterprise and private networks.
- **Erigon:** A Go implementation focused on archive node efficiency — stores the full history with much less disk space than Geth.

Consensus Layer Clients

- **Prysm:** The most widely used consensus client, written in Go by Prysmatic Labs.
- **Lighthouse:** A Rust implementation by Sigma Prime — praised for performance and safety.
- **Teku:** A Java implementation by ConsenSys — popular for enterprise staking.
- **Nimbus:** A Nim implementation by Status — designed for resource-constrained devices.
- **Lodestar:** A TypeScript implementation by ChainSafe — the only major client in JS/TS.

Why Client Diversity Matters

If one client has a bug and that client runs >66% of validators, the bug could cause the chain to finalise incorrectly — a catastrophic event.

Ethereum's community actively campaigns for validators to diversify across clients. The target is no single client running more than 33% of validators.

This is a direct lesson from Solana's outage history — where a single validator client bug took down the entire network. Ethereum's multi-client architecture is designed to prevent this.

06 — VALIDATORS, STAKING & BLOCK PRODUCTION

Becoming an Ethereum Validator

To become an Ethereum validator, a participant must deposit exactly **32 ETH** into the Ethereum deposit contract — a smart contract on the execution layer that registers validator credentials. This 32 ETH serves as the validator's stake — collateral that can be slashed if the validator behaves maliciously.

Why Exactly 32 ETH?

The 32 ETH requirement was chosen to balance two competing needs: low enough to allow a wide range of participants (not just institutions), but high enough to make attacks economically costly. At current prices (~\$3,000 per ETH), 32 ETH requires approximately \$96,000 — making solo staking accessible to committed individuals but not to the general public. This reality has driven massive growth in liquid staking protocols like Lido and Rocket Pool.

Validator Duties

DUTY	FREQUENCY	REWARD / PENALTY
Attest to the head of the chain	Every epoch (~6.4 min)	Attestation reward (largest source of income)
Propose a block	~Once per month (random)	Block proposal reward + priority fees
Participate in sync committees	~Once per year (random)	Sync committee reward
Vote on finality checkpoints	Every epoch	Finality reward
Stay online	Continuous	Inactivity leak penalty if offline

Staking Returns

Ethereum validators earn staking rewards from two sources: protocol-issued rewards (new ETH) and priority fees from transactions. The total APY varies based on the number of active validators — as more validators join, individual rewards decrease, since the same reward pool is shared among more participants.

TOTAL VALIDATORS	APPROX. ANNUAL YIELD (PROTOCOL)
500,000	~5.2% APY
700,000	~4.4% APY

900,000	~3.9% APY (approx. current, 2025)
1,000,000	~3.7% APY (projected)

Slashing Conditions

Slashing is the permanent partial destruction of a validator's staked ETH as punishment for provably malicious behaviour. Unlike Solana's conservative slashing, Ethereum's slashing is well-defined and actively enforced. There are two categories of slashable offences:

- **Double voting (equivocation):** Signing two different blocks for the same slot, or two different attestations for the same target. This is the most common slashing scenario.
- **Surround voting:** Casting an attestation that 'surrounds' a previous attestation — a vote that contradicts an earlier vote in a way that could support a conflicting fork.

When a validator is slashed, an initial penalty of 1/32 of its stake is immediately deducted, it is forced into an exit queue, and a further 'correlation penalty' is applied at the time of exit — proportional to how many other validators were slashed around the same time. If many validators are slashed simultaneously (suggesting a coordinated attack), the correlation penalty can reach up to 100% of the stake.

Liquid Staking: Lido & Rocket Pool

For users who cannot afford 32 ETH or do not want to run their own validator, **liquid staking protocols** offer a solution. Users deposit any amount of ETH and receive a liquid staking token in return — representing their staked ETH plus accruing rewards — which can be used freely in DeFi.

PROTOCOL	TOKEN	ETH STAKED	MODEL	NOTES
Lido	stETH	~9M ETH	Permissioned node operators	Largest; ~28% of all staked ETH; decentralisation concerns
Rocket Pool	rETH	~800K ETH	Permissionless — anyone can run a node	Most decentralised liquid staking option
Coinbase	cbETH	~600K ETH	Centralised — Coinbase operates validators	Easiest for Coinbase users; centralised risk
Frax	sfrxETH	~250K ETH	Protocol-owned validators + DeFi integration	Highest yields via DeFi strategies

The Lido Centralisation Concern

Lido Finance controls approximately 28-30% of all staked ETH — giving it potential influence over Ethereum's consensus.

If Lido's node operators were to coordinate maliciously (or be compelled to by a government), they could represent a meaningful fraction of the 33% needed to halt finality.

This is widely recognised as Ethereum's most pressing decentralisation challenge. The community has debated whether to implement protocol-level limits on liquid staking concentration.

This concludes File 1 of the Ethereum Deep-Dive Research Series — prepared by **Nasir Bello**.

File 2 covers the ETH token, DeFi ecosystem, NFTs, partnerships, and milestones.

File 3 covers Layer 2s, competitive landscape, future roadmap, and conclusion.